



Welcome to

AASD 4011: Deep Learning II

Credit hours: 48

Instructor: Dr. Moe Fadaee

Email: mo.fadaee@georgebrown.ca

George Brown College

Agenda:

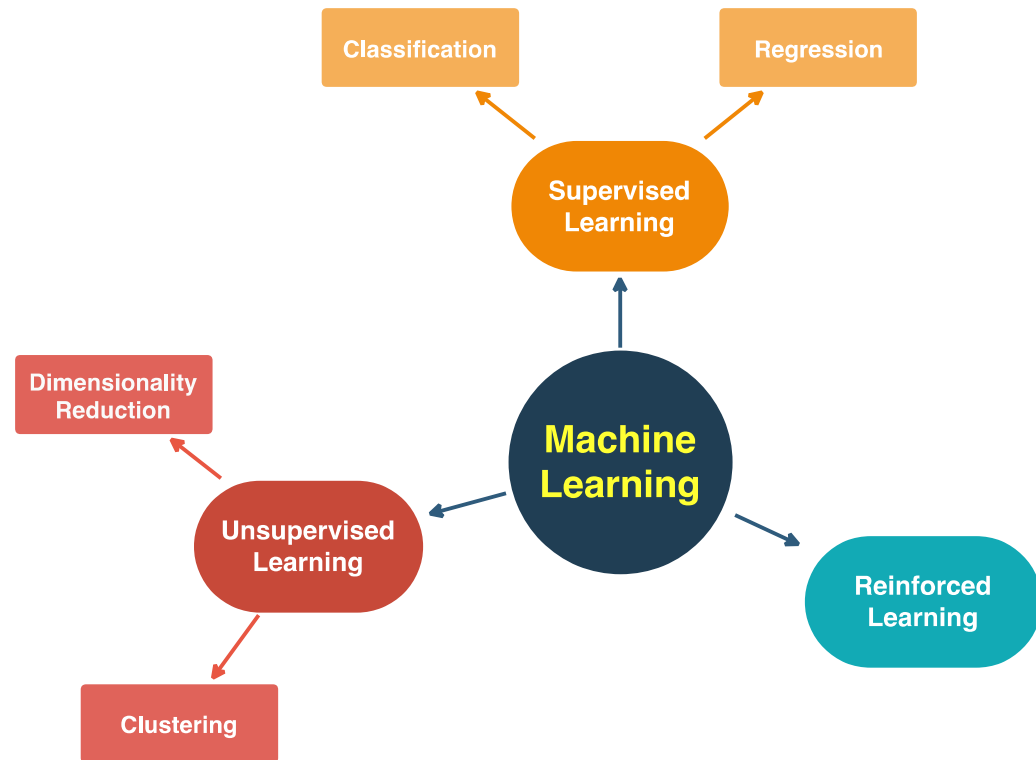
1. Features maps
2. Object detection
3. SSD
4. R-CNN, Fast R-CNN, Faster R-CNN

Machine Learning subcategories:

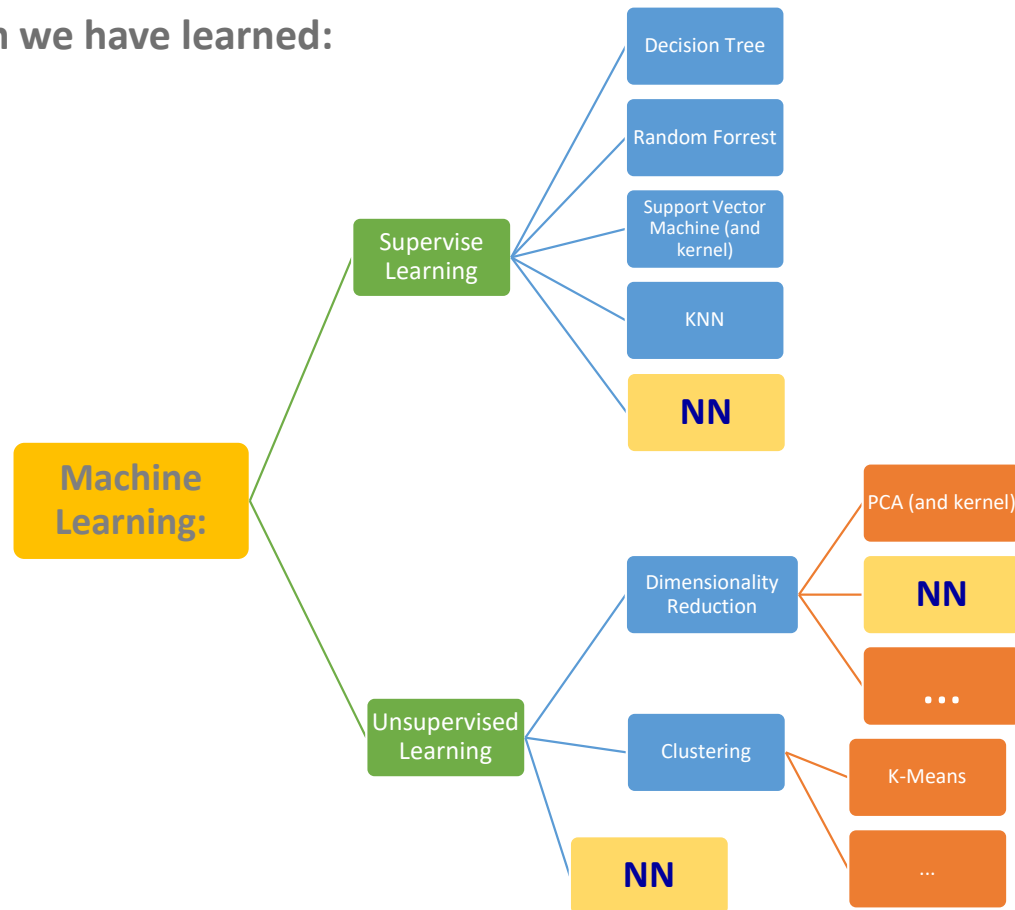
- I. Supervised Learning
- II. Un-supervised Learning
- III. Reinforced learning
- IV. Others...

Where is NN:

Almost all of them



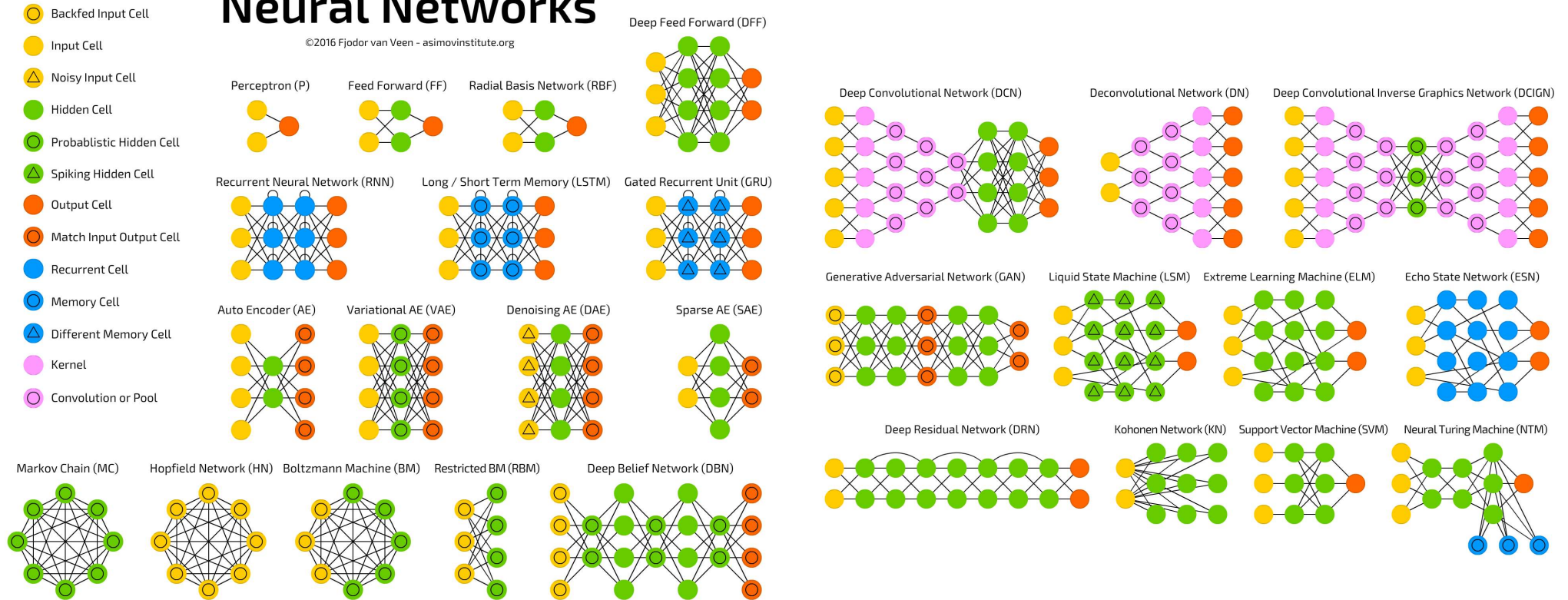
What algorithm we have learned:



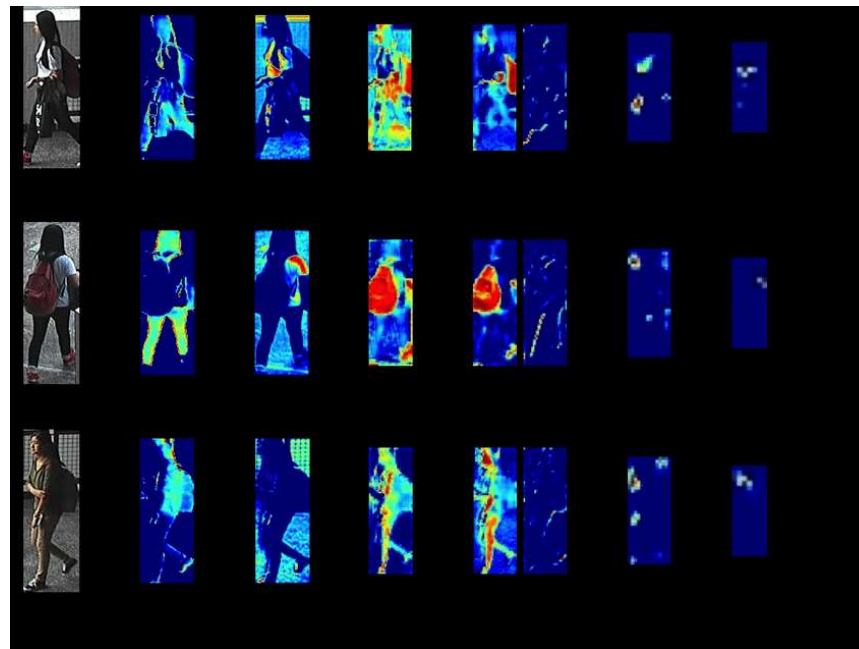
Different structures of Neural Networks:

A mostly complete chart of Neural Networks

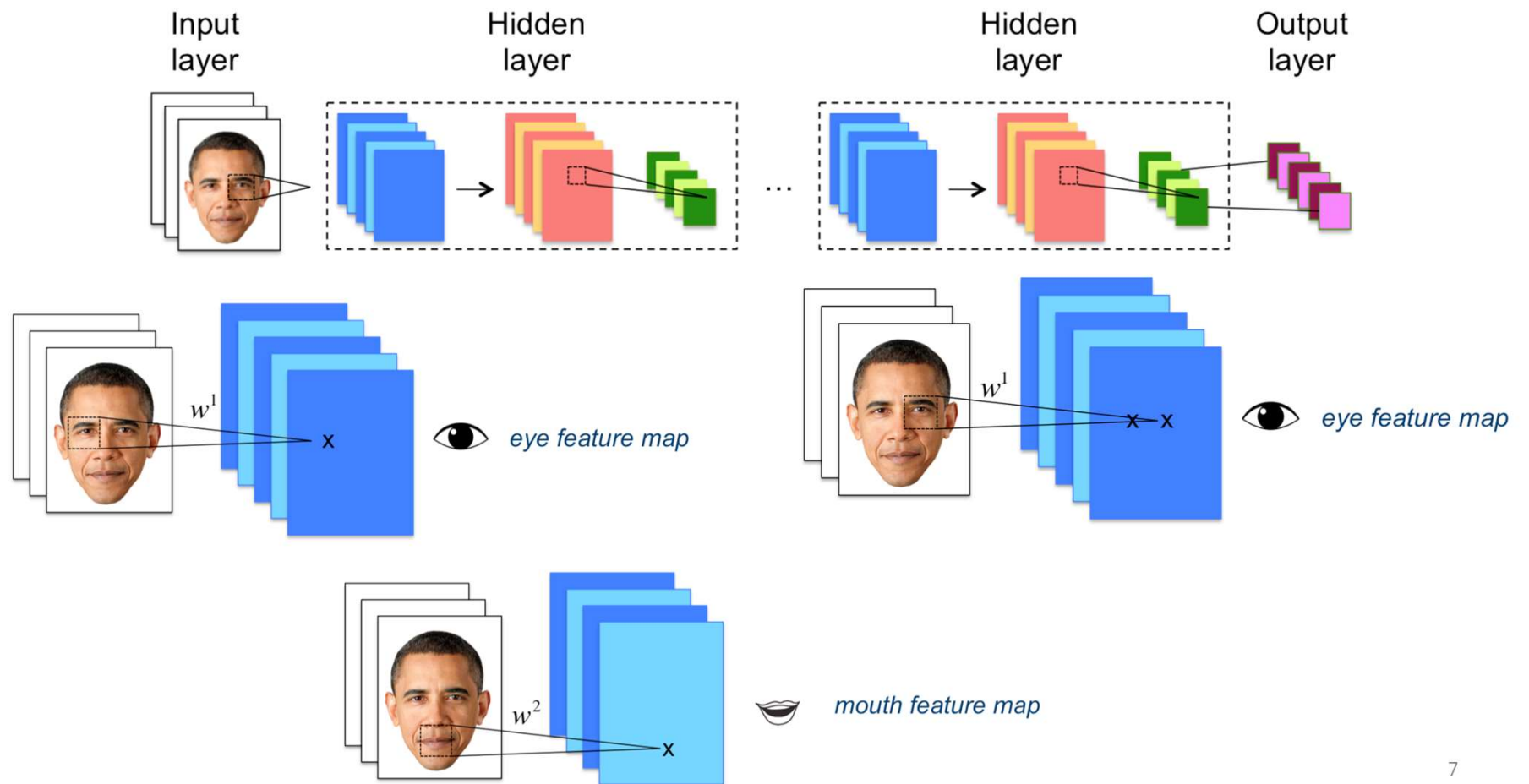
©2016 Fjodor van Veen - asimovinstitute.org



Feature maps



Feature maps: lets look at Convolutions again



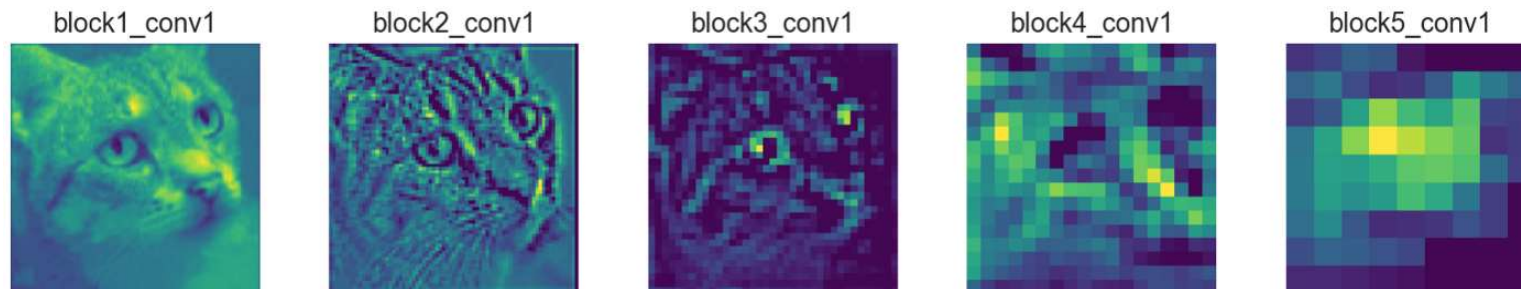
Feature maps:

Feature map or activation map

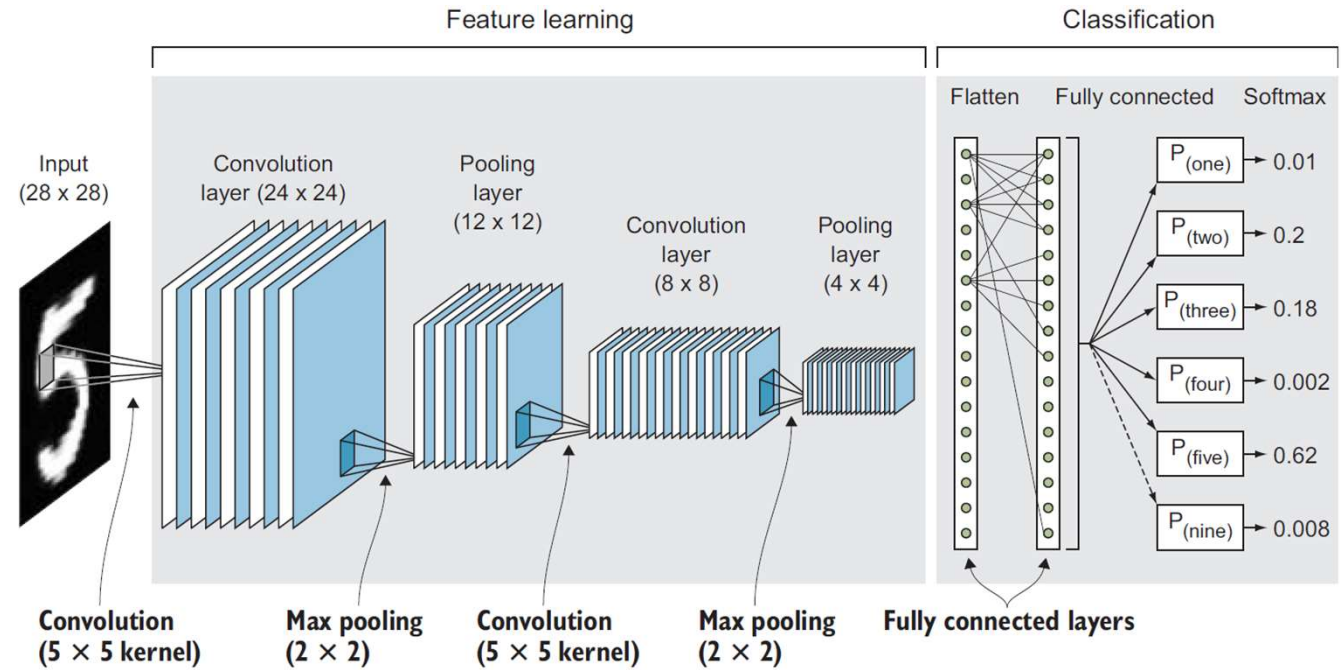
It is a mapping that corresponds to the activation of different parts of the image (like a noise), and also a feature map because it is also a mapping of where a certain kind of feature is found in the image (remember filters or kernels are there to detect certain features).

Where we have a high activation it means certain features are found.

How to extract feature maps? By using Convolutional Neural Networks



Feature maps:



CNN model architectures:

There are several CNN model architecture out there, that are already pre-trained.

Few examples: VGG, AlexNet, Mobilenet...

Different CNN models:

<https://vitalflux.com/different-types-of-cnn-architectures-explained-examples/>

Feature maps:

e.g. VGG architecture

```
model = Sequential()

model.add(Conv2D(input_shape=(224,224,3),filters=64,kernel_size=(3,3),padding="same", activation="relu"))

model.add(Conv2D(filters=64,kernel_size=(3,3),padding="same", activation="relu"))

model.add(MaxPool2D(pool_size=(2,2),strides=(2,2)))

model.add(Conv2D(filters=128, kernel_size=(3,3), padding="same", activation="relu"))

model.add(Conv2D(filters=128, kernel_size=(3,3), padding="same", activation="relu"))

model.add(MaxPool2D(pool_size=(2,2),strides=(2,2)))

model.add(Conv2D(filters=256, kernel_size=(3,3), padding="same", activation="relu"))

model.add(Conv2D(filters=256, kernel_size=(3,3), padding="same", activation="relu"))

model.add(Conv2D(filters=256, kernel_size=(3,3), padding="same", activation="relu"))

model.add(MaxPool2D(pool_size=(2,2),strides=(2,2)))

model.add(Conv2D(filters=512, kernel_size=(3,3), padding="same", activation="relu"))

model.add(Conv2D(filters=512, kernel_size=(3,3), padding="same", activation="relu"))

model.add(Conv2D(filters=512, kernel_size=(3,3), padding="same", activation="relu"))

model.add(MaxPool2D(pool_size=(2,2),strides=(2,2)))

model.add(Conv2D(filters=512, kernel_size=(3,3), padding="same", activation="relu"))

model.add(Conv2D(filters=512, kernel_size=(3,3), padding="same", activation="relu"))

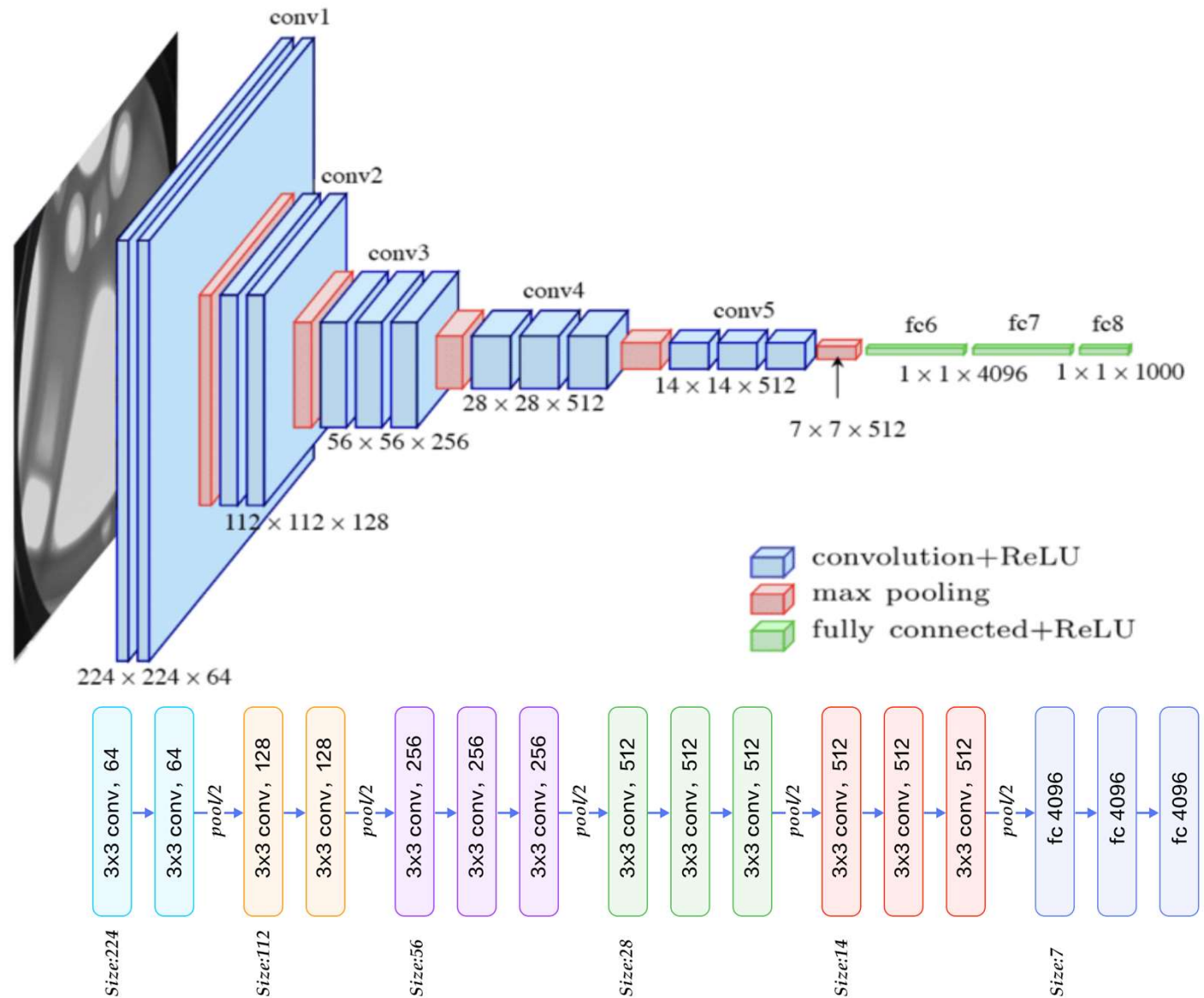
model.add(Conv2D(filters=512, kernel_size=(3,3), padding="same", activation="relu"))

model.add(MaxPool2D(pool_size=(2,2),strides=(2,2)))
```

Feature maps:

CNN model architectures:

e.g. VGG:

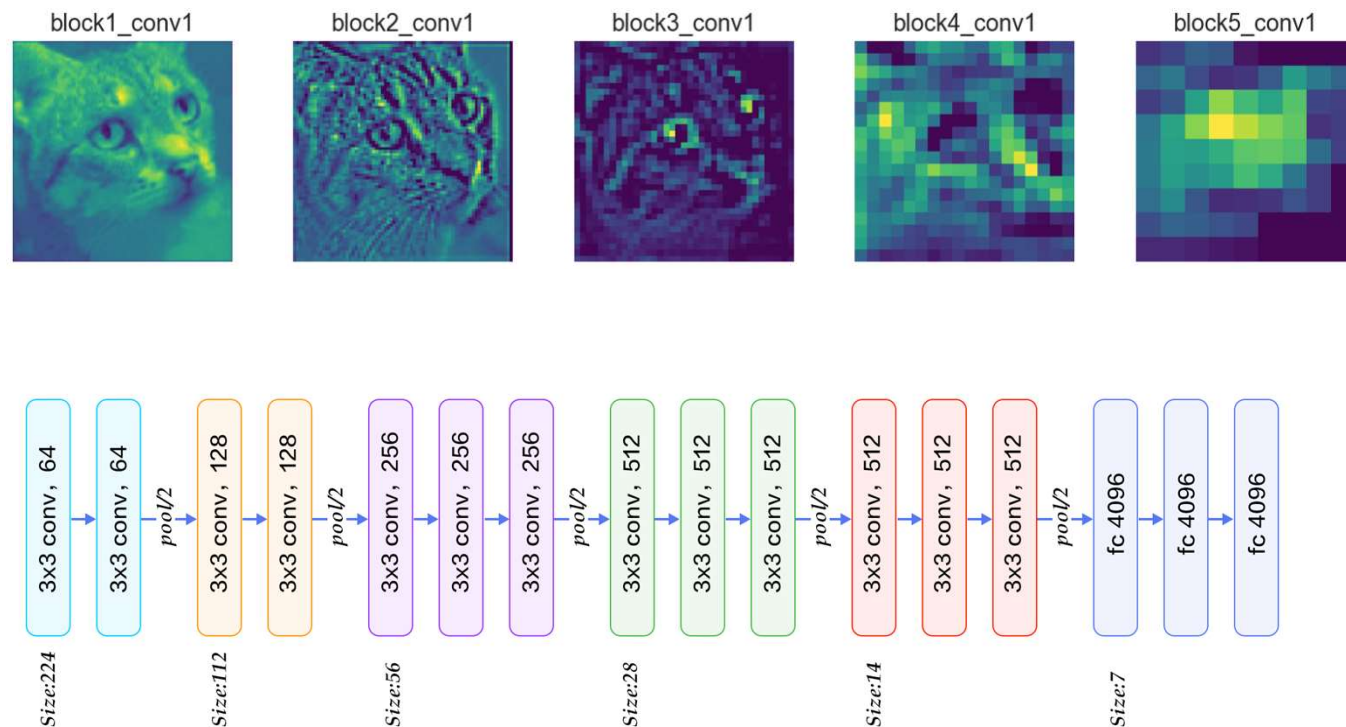


Feature maps: What happens?

Few first layer are responsible to detect simple (low level) features (lines, edges, color...)

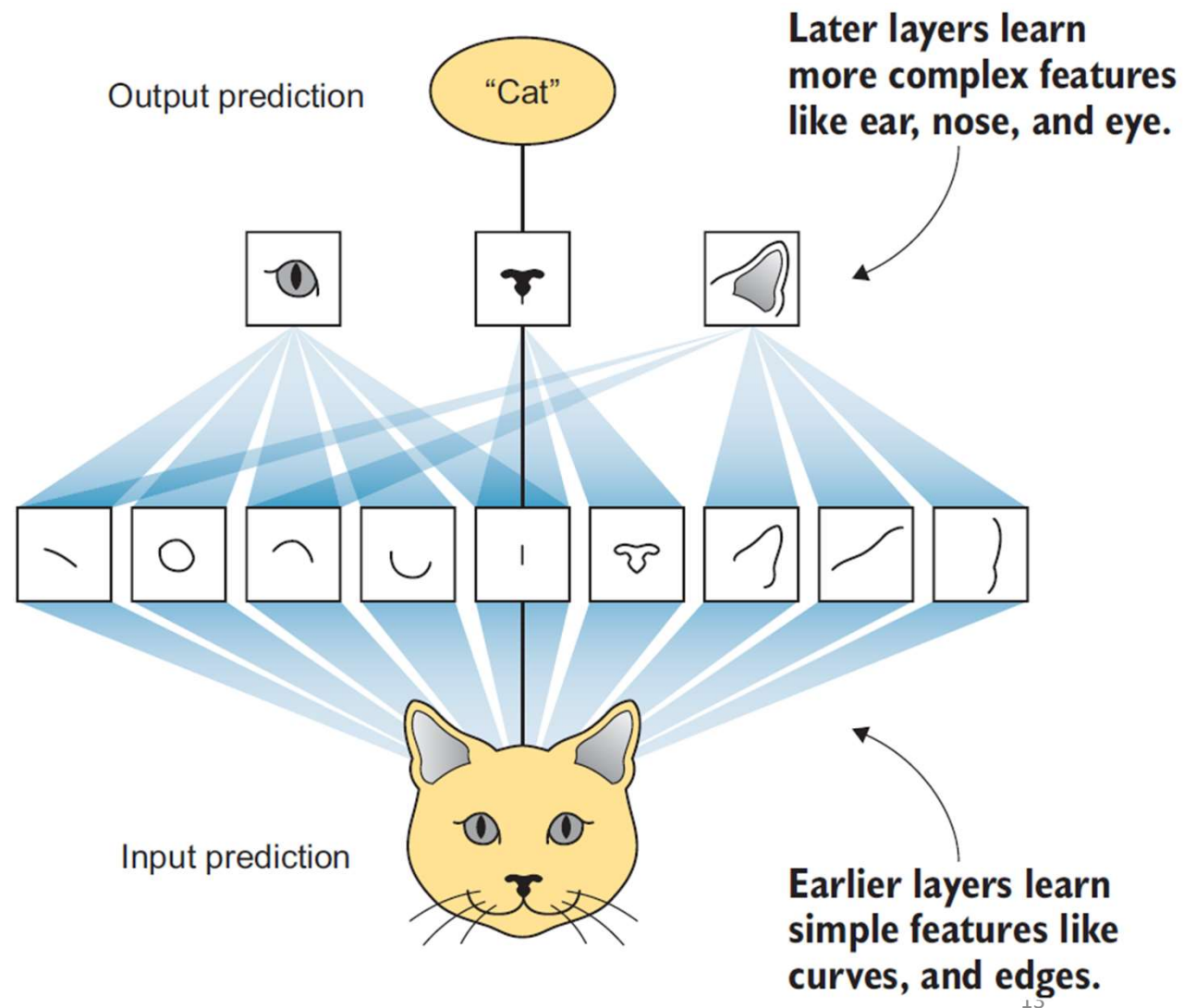
As we go deeper, the images are a representative of original image, the deeper layers only focus on high level features (like cat eyes).

The last images are mainly for classification and are not meaningful to us any more



Feature maps:

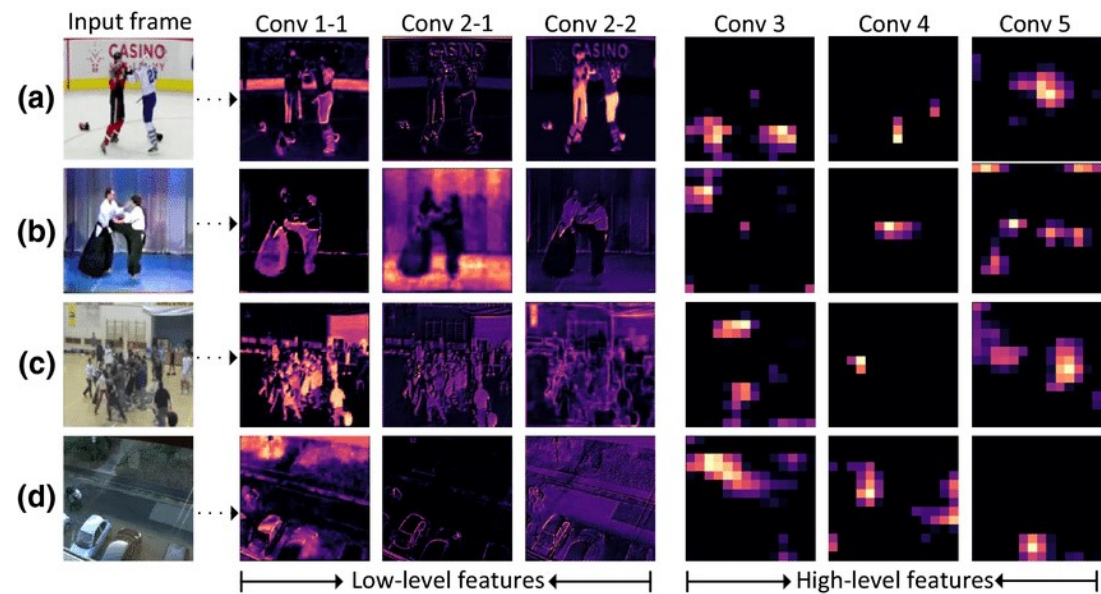
- An eye filter get activated by input that looks like an eye
- It only activated by activation maps of the previous layers that that correspond to the eye in the input image



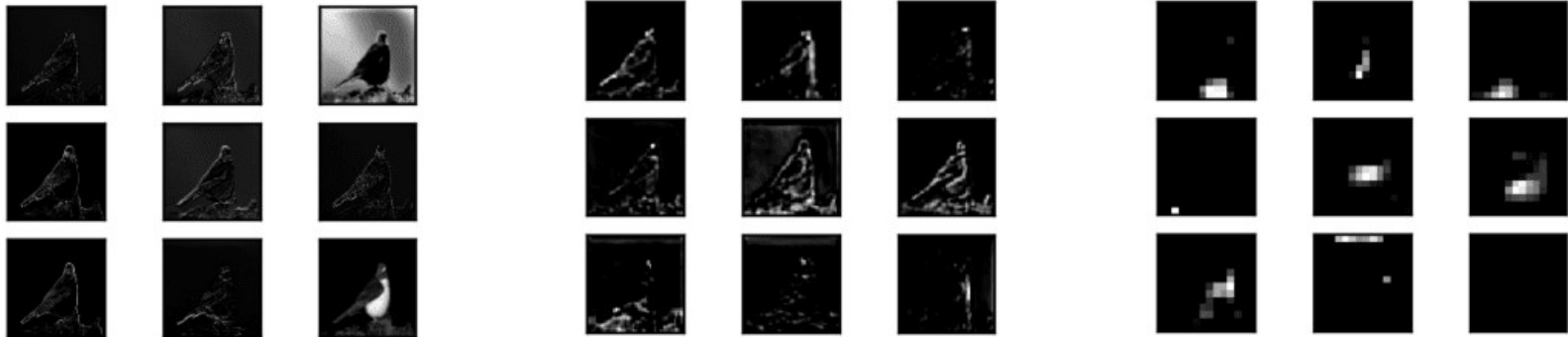
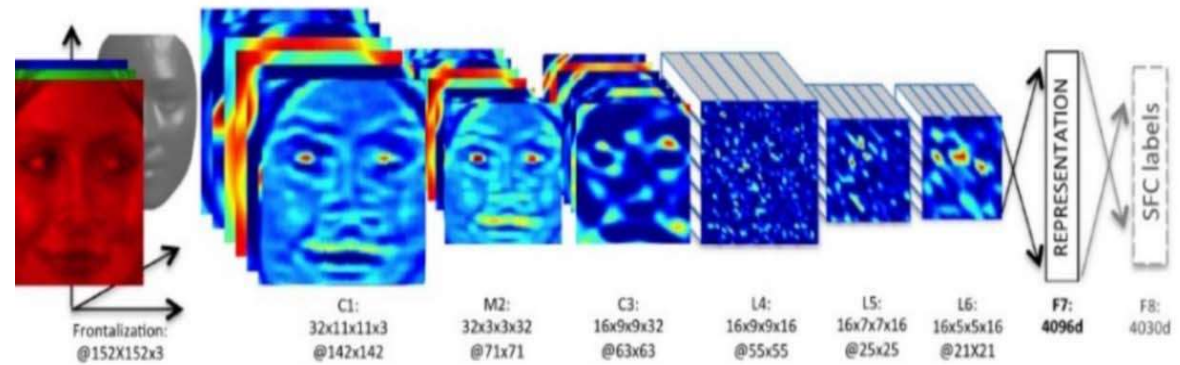
Feature maps:

Another example,

Think of this as an object from a distance far getting closer and closer to you (what do you see first, what do you see few sec after and ...)



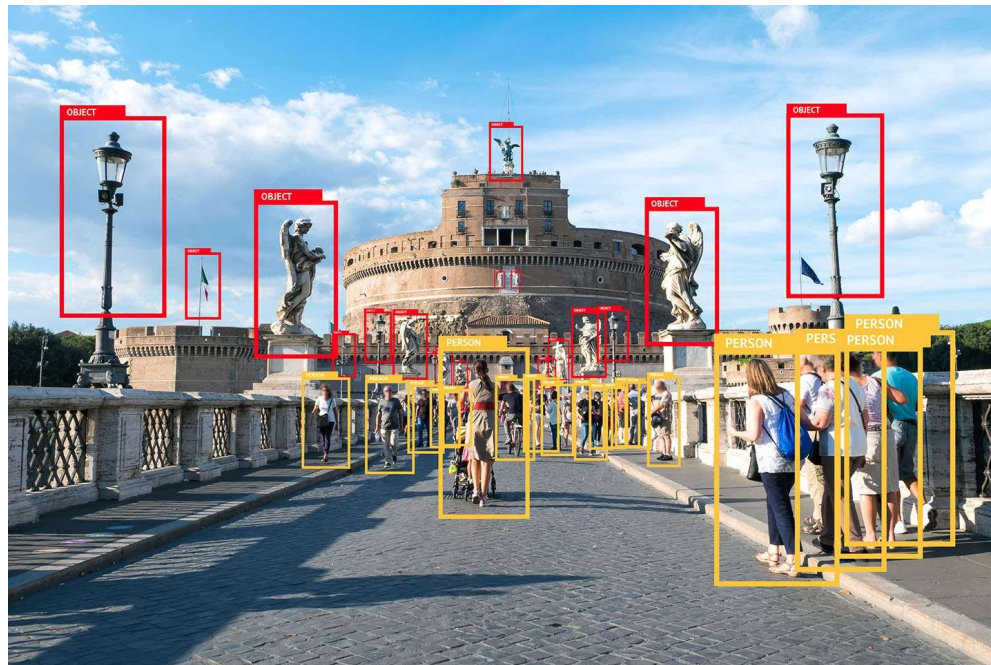
Feature maps:



Source:

<https://machinelearningmastery.com/how-to-visualize-filters-and-feature-maps-in-convolutional-neural-networks/>

Object Detection



How would you build an Object detection model:

What about using a sliding window approach?

Basically have a sliding window to go over the image and do a classification on each.

What are the problems?

1. What should be the shape of the window?
2. It is so time consuming



Object detections:

Different algorithms

- 1. YOLO (You Only Look Once)**
- 2. SSD (Single Shot Detector)**
- 3. R-CNN (Region-based Convolutional Neural Networks)**
- 4. R-FCN (Region-based Fully Convolutional Networks)**
- 5. and more...**

Object detections: Different algorithms

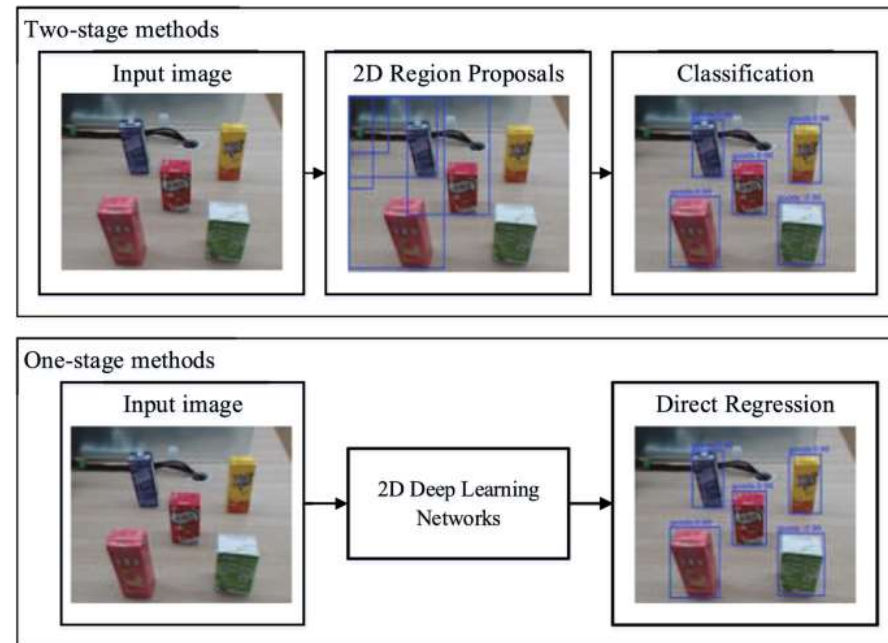
They are generally 2 types of Object Detection algorithms:

1. Classification based Algorithms (2 stage detectors)

- First they will find a regions of interest (ROI) and then classify these regions to different categories,
- Example R-CNN
- Usually more accurate

2. Regression based Algorithms (1 stage detectors)

- These algorithms predict classes and bounding boxes for the whole image at once
- Examples: SSD, YOLO
- Usually faster than region based models



SSD (Single Shot Detector)

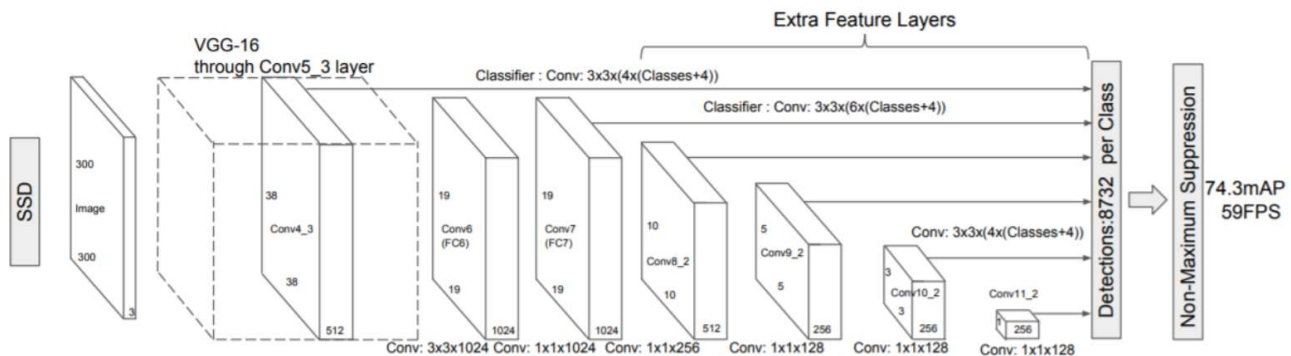
Object detections: SSD

Paper: <https://arxiv.org/pdf/1512.02325.pdf>

SSD use a fully convolutional approach in which the network is able to find all objects within an image at once ('single-shot' or 'look once') through the convolutional networks.

SSD has 2 components:

1. The backbone
2. SSD head



Object detections: SSD

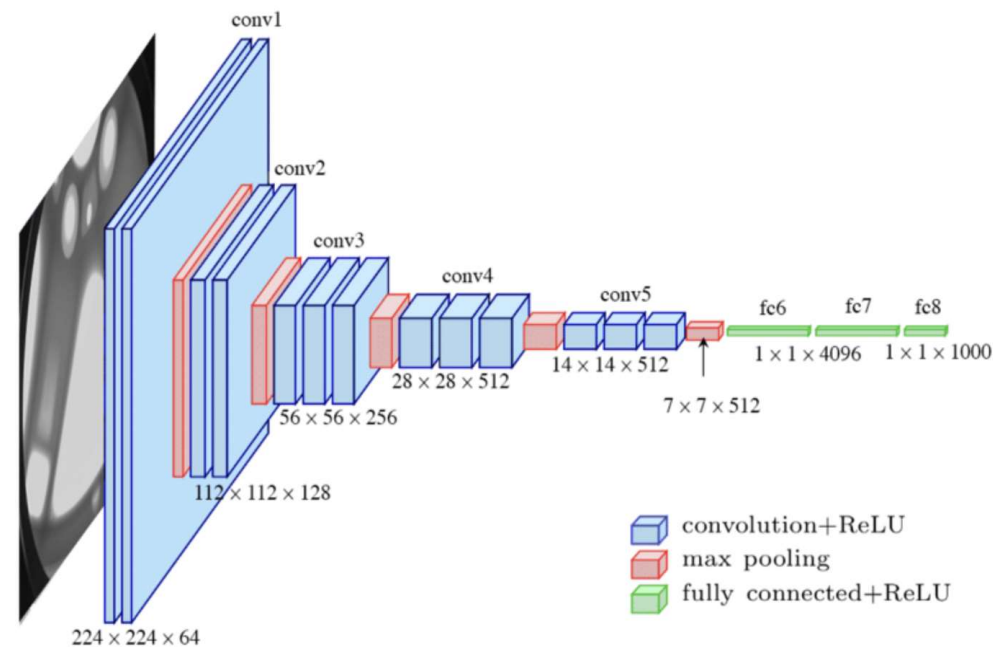
The Backbone component:

The backbone which usually a pretrained model is responsible for detections of feature maps (what we mean here?). The final classification layers are removed to be able to extract feature maps.

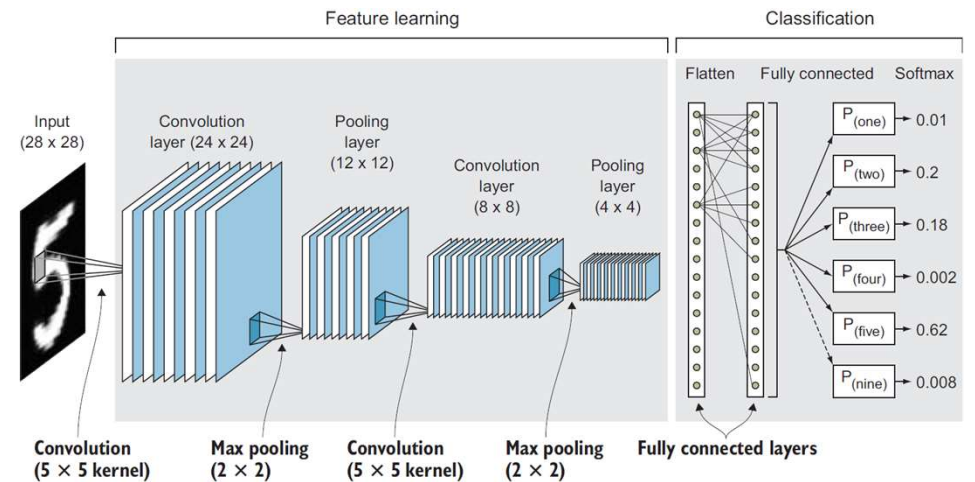
This can be a Resnet, VGG...

As an example how many feature maps this VGG gives you?

X feature maps that are different representative of the input image

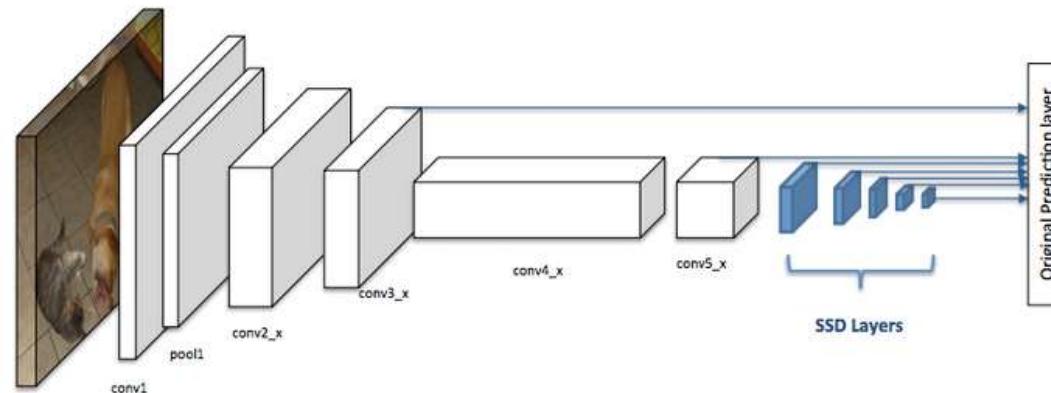


Object detections: SSD



The SSD component:

It refers to extra convolution layers that will replace the classification component in the CNN architecture. These layers are responsible for generating the bounding boxes and the classes (with probabilities).



Object detections: SSD

Grid Cells

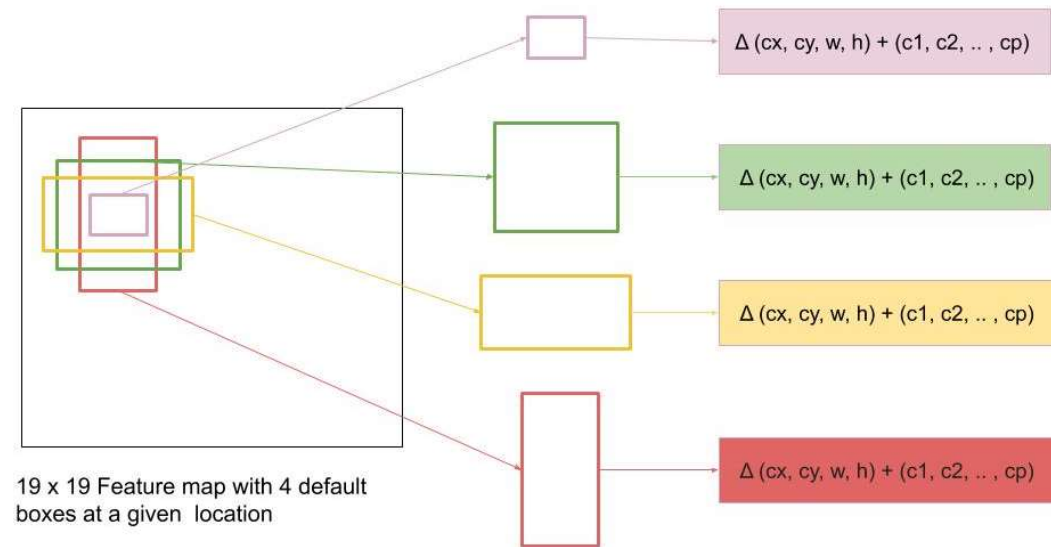
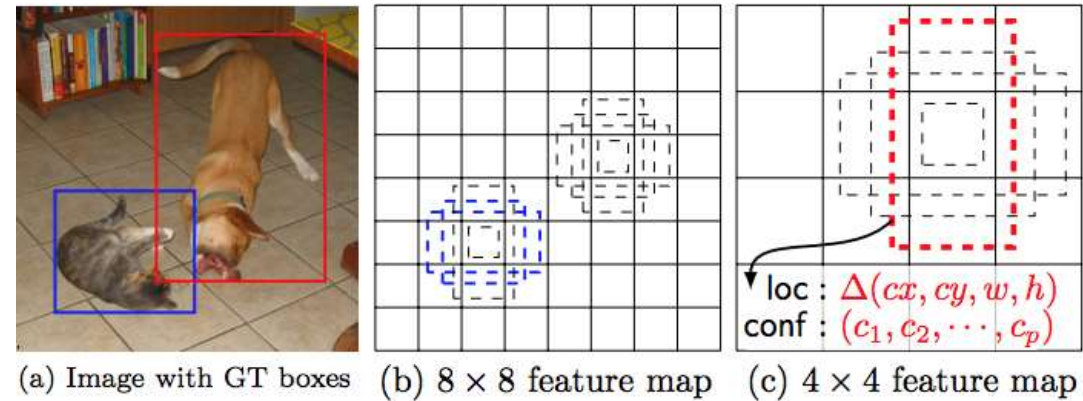
Different feature maps with different sizes (resolutions) are taken from different layers of the CNN.

For each cell of the feature maps, default bb (anchor boxes) will be created, these have a fix location wrt the cell.

For each of the cells, SSD predict the bb offsets (coordinates with respect to the cell itself $\Delta(cx, cy, w, h)$), to better match the ground truth. In addition to the bb coordinates, for the every cell, SSD also produce a confidence score for each of the classes ($c_1, c_2, \dots, c_p$).

In other word, the object detection (predicting the class and location of an object) will be performed for each cell, this is done for different classes with bounding boxes of various aspect ratio.

If no object is detected that cell is ignored.



For each of the 4 default boxes predict 4 offsets relative to the default box shape, plus class scores for p categories. This outputs $19 \times 19 \times 4 \times (c + 4)$ for the entire feature map.

Object detections: SSD

Why multiple bb, with different sizes?

Why several feature maps?

Object detections: SSD

Why multiple bb, with different sizes?

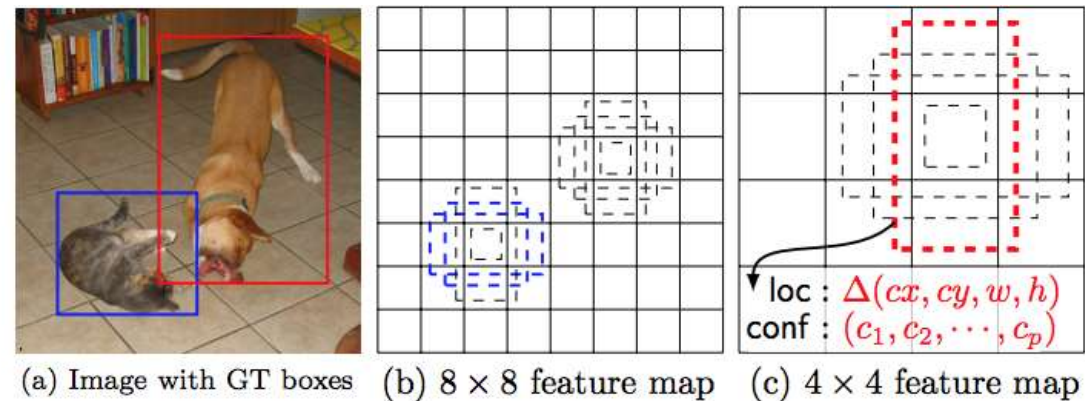
Why several feature maps?

This is to capture various objects with different sizes. Why?

Object detections: SSD

Why multiple bb, with different sizes?

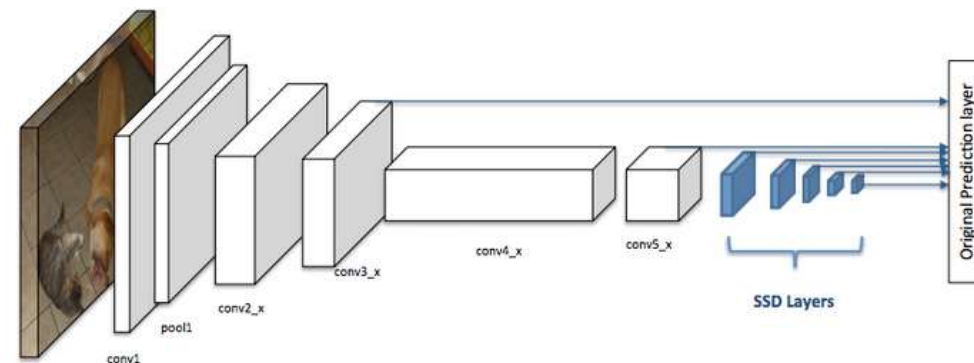
Why several feature maps?



This is to capture various objects with different sizes. Why?

Larger objects are presented in the lower resolution feature maps (think of down-sampling, and how a larger object will fit into a lower res feature map). The small objects are almost lost in the upper layers of the NN.

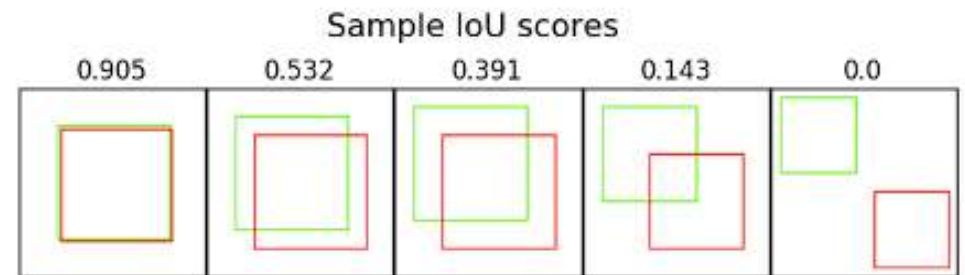
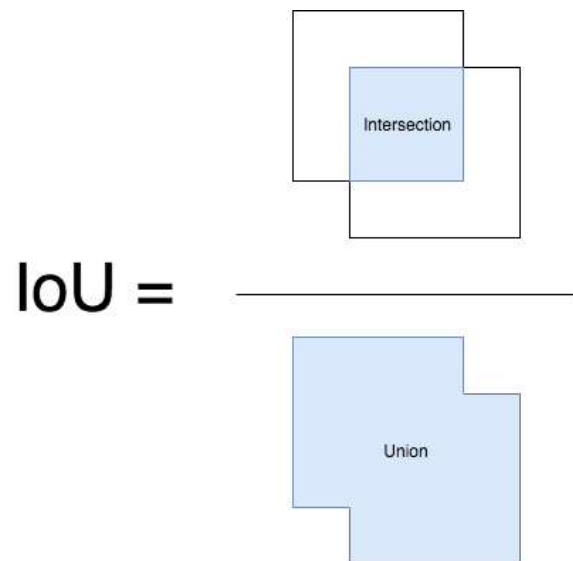
In the example above cats got captured in the 8x8 map and the dog in the 4x4 map, these maps got collected from different level of the CNN.



Object detections: SSD

Intersection over Union (IoU) or Jaccard index:

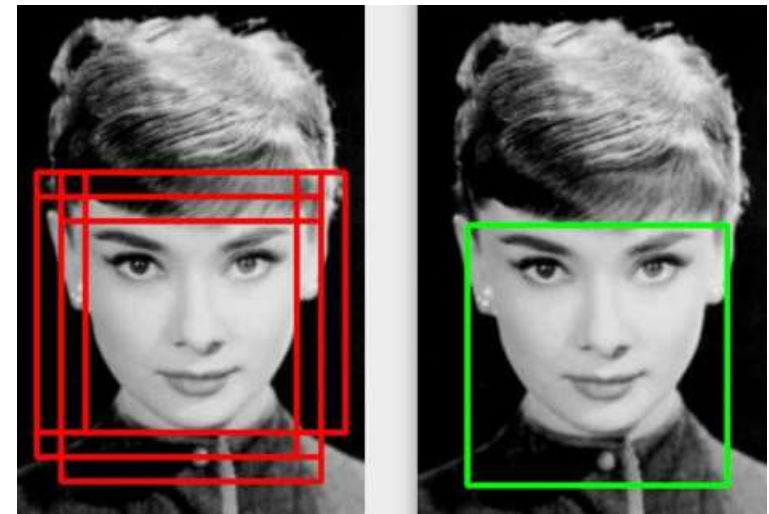
$\text{IoU} = \text{Area of the intersection} / \text{Area of the union}$



Object detections: SSD

Non-max suppression

1. Input:
 1. take a list of all bounding box (bb) proposals (list **BB**)
 2. with their confidence score
 3. and also a threshold for IoU, th_{iou}
2. Take the box with highest score from **BB**, (call this bb_1)
3. Remove bb_1 from **BB** and add it to new list **S_BB**
4. Calculate IoU between bb_1 and all other bb in **BB**
5. Remove all bb that has a IoU larger than th_{iou}
6. Repeat step 2 to 5, until no more bb left in **BB**



R-CNN
(Region-Based CNN)

Object detections: R-CNN (Regime-Based CNN)

R-CNN

Region based CNN models uses selective search to extract region of interests from images and only work with this RoI (region of interest) rather than a large number of boxes.

Original paper: <https://arxiv.org/abs/1311.2524>

Selective search do this by over-segmenting the image. But these segmented parts wont be readily available to be taken as RoI (as an example look at objects with several segments, or one segment with several objects within it).

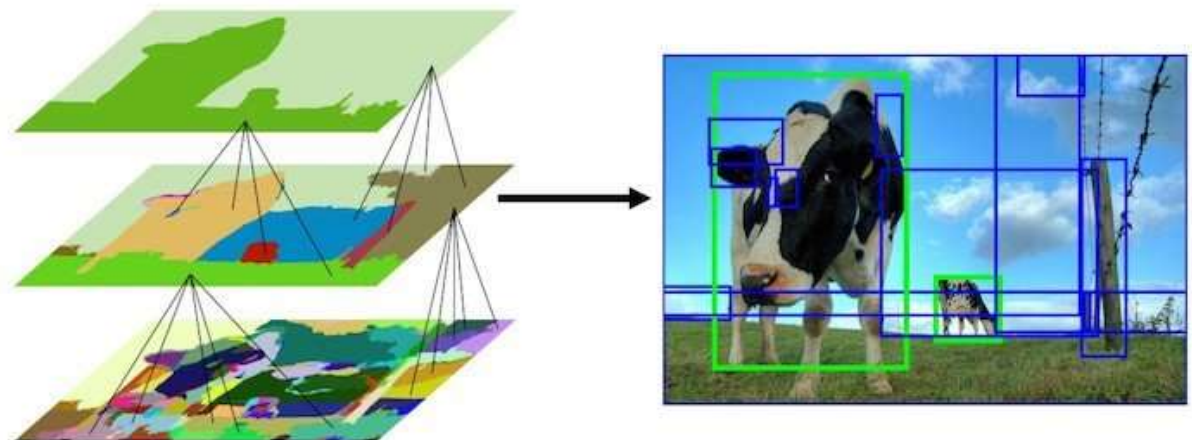
Regular segmentation vs over-segmenting



Object detections: R-CNN (Region-Based CNN)

Selective search

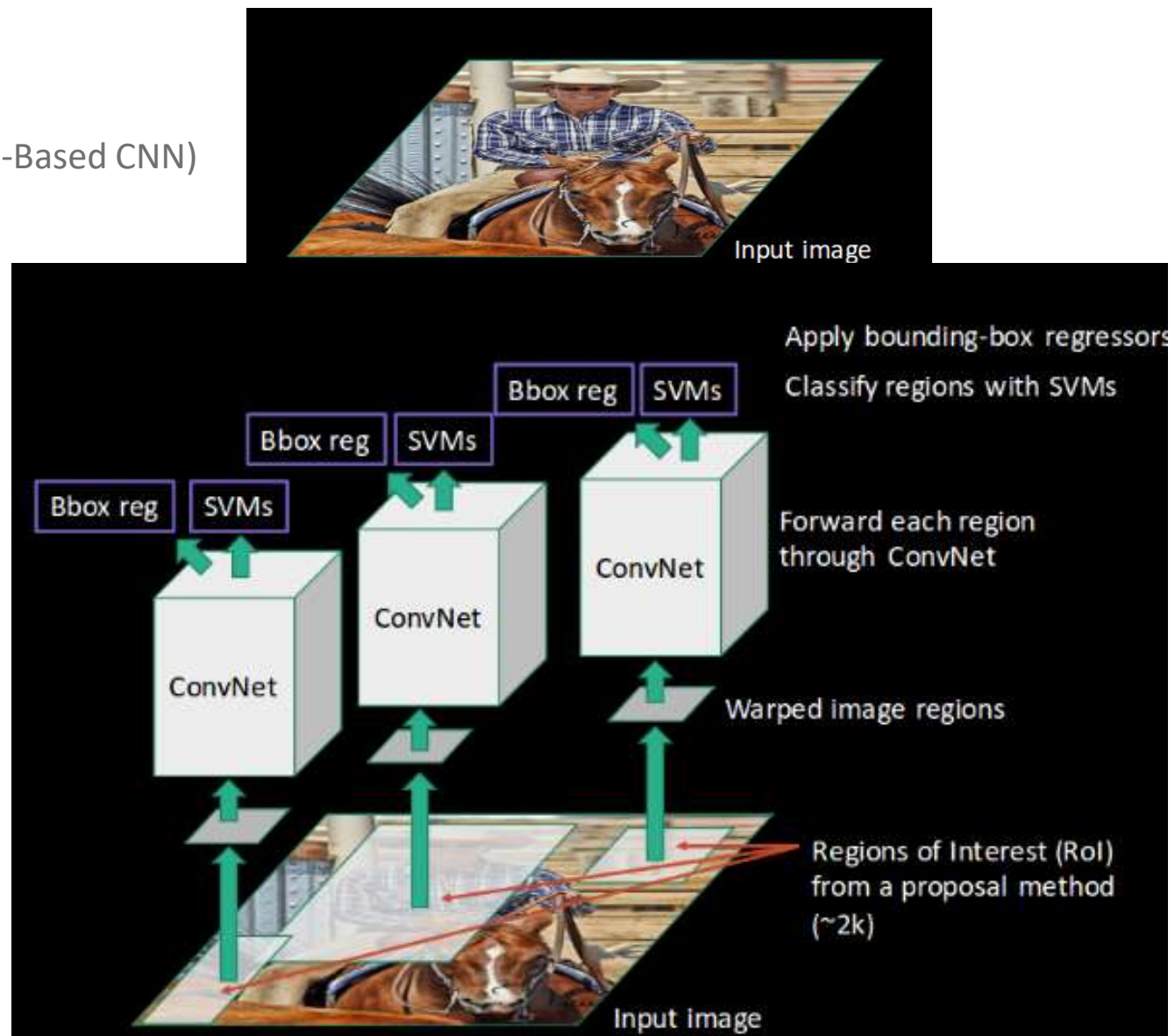
1. Perform over-segmentation
2. Find area of interests within the segments (find BB)
3. Add these BBs to the list of RoI
4. Group adjacent segments
5. Redo step 2 to 5



Object detections: R-CNN (Regime-Based CNN)

R-CNN

How R-CNN works



Object detections: R-CNN (Regime-Based CNN)

R-CNN: the problem

Object detections: R-CNN (Regime-Based CNN)

R-CNN: the problem

We need to run the CNN on so many RoI

Object detections: R-CNN (Regime-Based CNN)

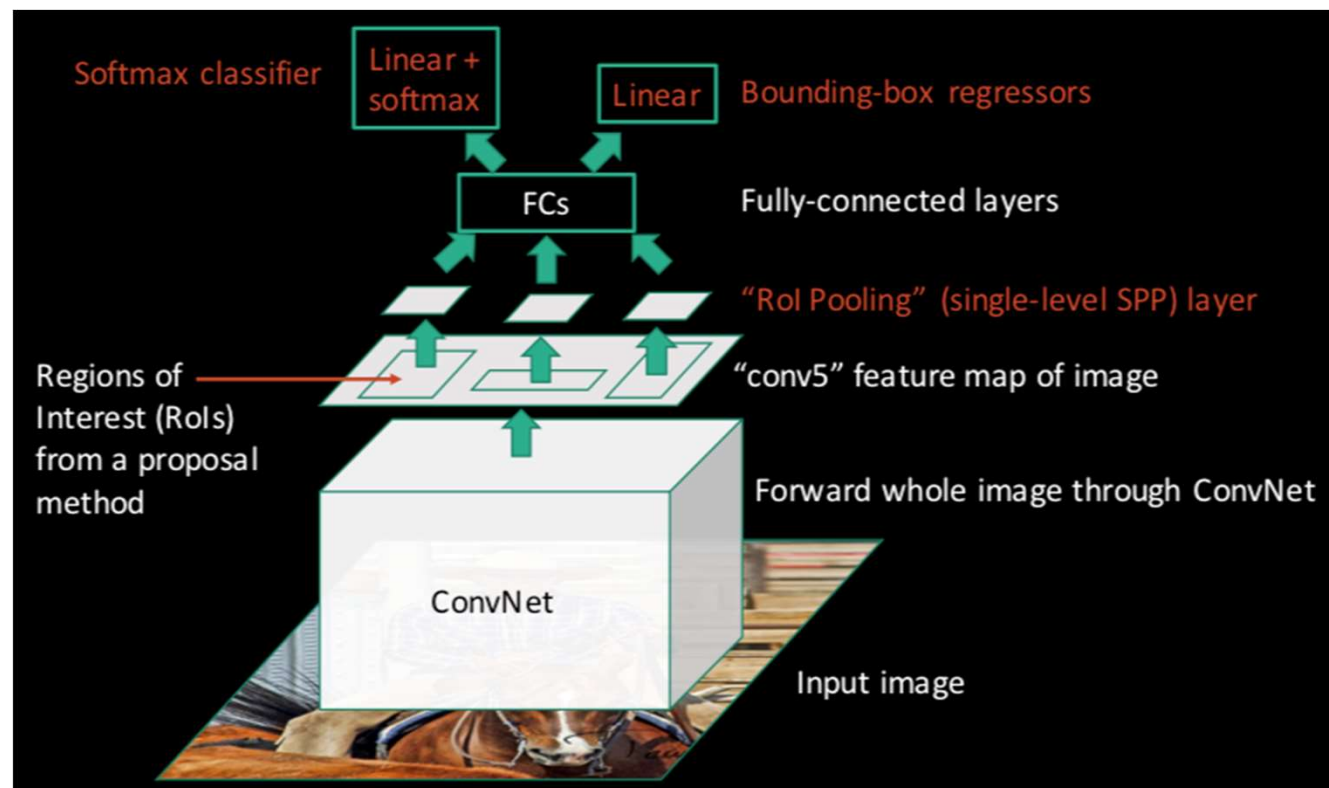
Fast R-CNN

Original paper:

<https://arxiv.org/abs/1504.08083>

In Fast R-CNN, the CNN is run on the whole image at once. CNN is used to extract feature maps. Selective search is done as usual, RoI generated by selective search will be projected on feature maps.

RoI are then go through a pooling layer to be converted to a fixed size, and then feed into a FC layer for classification.



Object detections: R-CNN (Regime-Based CNN)

Fast R-CNN: the problem

Object detections: R-CNN (Regime-Based CNN)

Fast R-CNN: the problem

Fast R-CNN is still slow, as it still uses selective search.

Object detections: R-CNN (Regime-Based CNN)

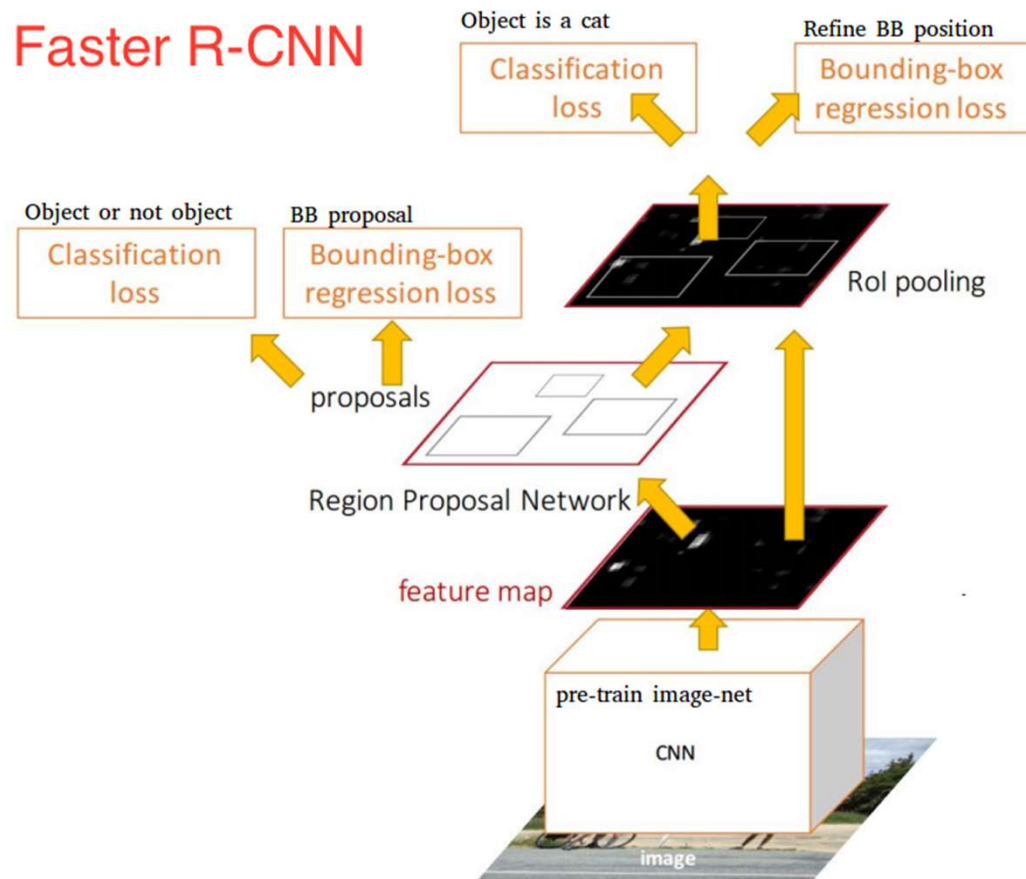
Faster R-CNN:

Original paper:

<https://arxiv.org/abs/1506.01497>

Faster R-CNN is a combination of Fast R-CNN and RPN (Region Proposal Network).

Faster R-CNN



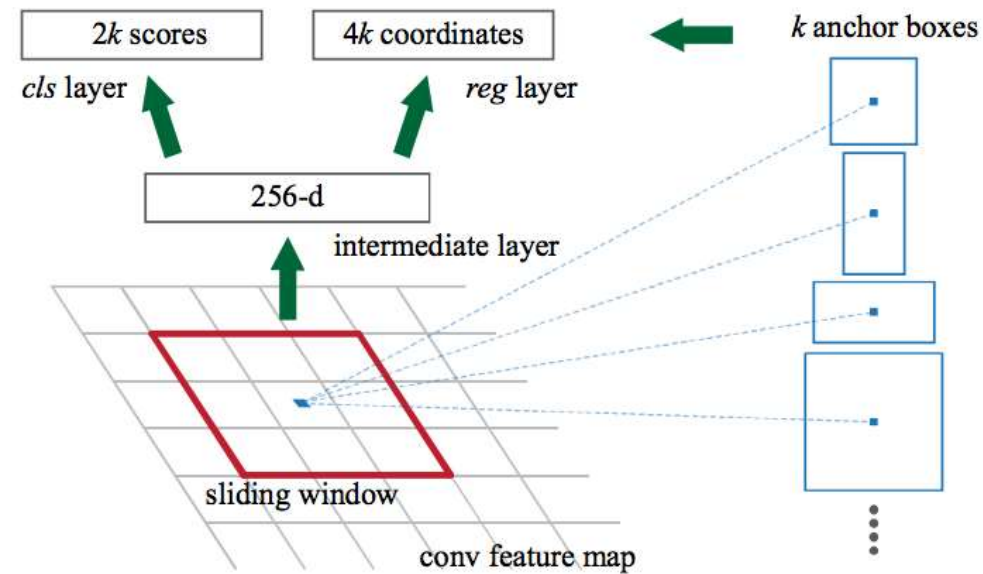
Object detections: R-CNN (Regime-Based CNN)

Faster R-CNN:

RPN (Region Proposal Network):

RPN do both classification and regression. It uses a sliding window to go over the feature map, for each sliding window it uses k anchor boxes. For each anchor box it predicts:

1. The probability of this box belonging to a class
2. The size adjustment so how this better fit the object



Object detections:

Several other methodologies:

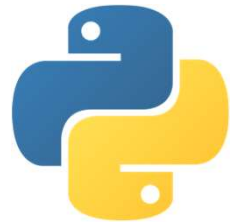
<https://www.saagie.com/blog/object-detection-part1/>

[https://amberer.gitlab.io/papers in ai/object detection](https://amberer.gitlab.io/papers_in_ai/object_detection)

Object detections: Performance Tuning

Different approaches to improve model performance

1. Add more images
2. Train for longer (more epoch)
3. Tune the architecture



Python practice :

Please follow the instruction provided on the TF object detection API, to build your first TF OD models.